

🔗 See Topic 16 → Intervals Masterclass

🎯 **GOAL:** Think in Binary! Master the 8 Fundamental Bit Tricks, XOR Cancellation Laws, Bitmask Subset Enumeration, Bitwise Addition ('Sum of Two Integers'), and solve all NeetCode Bit Manipulation questions!

🔧 REUSABLE BITWISE UTILITIES

Core Bit Operations:

```
// Clear lowest set bit: n & (n - 1)
n = n & (n - 1);
// Isolate lowest set bit: n & -n
int lowestBit = n & -n;
// Check if i-th bit is set: (n >> i) & 1 == 1
boolean isSet = ((n >> i) & 1) == 1;
```

Bitmask Subset Enumeration:

```
// Iterate over all submasks of a bitmask
for (int sub = mask; sub > 0; sub = (sub - 1) & mask) {
    // Process submask
}
```

💡 **AHA MOMENT #1: HARDWARE-LEVEL \$O(1)\$ CONSTANT TIME EXECUTION**

Bitwise operations execute directly on the CPU's Arithmetic Logic Unit (ALU) in a SINGLE clock cycle! They bypass memory allocation and object creation, running up to ****\$100\times\$ faster**** than standard data structure operations!

1 TWO'S COMPLEMENT REPRESENTATION

In Java, 32-bit signed integers use **Two's Complement** to represent negative numbers:

Formula: `-x = ~x + 1`

- To negate 'x', invert all bits (~x) and add '1'!
- Lowest Set Bit Trick ('x & -x')**: Exploits Two's Complement to isolate the rightmost 1-bit instantly!

2 ARITHMETIC VS UNSIGNED RIGHT SHIFT

Operator	Name	Behavior for Negative Numbers
>>	Arithmetic Right Shift	Fills left bits with Sign Bit (1s)
>>>	Unsigned Right Shift	Fills left bits with ZEROES (0s)

🔍 TWO'S COMPLEMENT VISUALIZATION (X = 6 → -X = -6)

BINARY TRANSFORMATION: Decimal 6 = 00000000 00000000 00000000 00000110 Invert (~6) = 11111111 11111111 11111111 11110001 Add 1 (+ 1) = 11111111 11111111 11111111 11110010 (-6 in Binary!) Lowest Set Bit (6 & -6): 00000000 00000000 00000000 00000110 (6) & 11111111 11111111 11111111 11110010 (-6) _____ = 00000000 00000000 00000000 00000010 (Decimal 2 - Isolates lowest set bit!)

🚀 **KEY TAKEAWAY:** Always use '>>>' (unsigned right shift) when shifting bit patterns where sign extension is unwanted!

- 💡 AHA MOMENT #2: THE 4 BITWISE TRUTH TABLES
- **AND** ('&'): '1' ONLY if BOTH bits are '1'. (Used for Masking / Checking).
 - **OR** ('|'): '1' if AT LEAST ONE bit is '1'. (Used for Setting bits).
 - **XOR** ('^'): '1' if bits are DIFFERENT ('1 ^ 0 = 1', '1 ^ 1 = 0'). (Used for Cancellation / Toggling).
 - **NOT** ('~'): Inverts every bit ('0 → 1', '1 → 0').

📊 BITWISE OPERATOR TRUTH TABLE MATRIX

A	B	A & B (AND)	A B (OR)	A ^ B (XOR)	~A (NOT A)
0	0	0	0	0	1
0	1	0	1	1	1
1	0	0	1	1	0
1	1	1	1	0	0

🚀 **KEY TAKEAWAY:** 'x ^ x = 0' and 'x ^ 0 = x' are the foundation of XOR-based cancellation algorithms!

**💡 AHA MOMENT #3: BRIAN KERNIGHAN'S ALGORITHM (`n & (n - 1)`)**

Subtracting `1` from `n` flips all trailing zeroes to `1`'s and the rightmost `1` to `0`.

Executing `n & (n - 1)` clears the lowest set bit in EXACTLY 1 operation! Repeating this counts set bits in $O(\text{set bits})$ iterations!

1 BASIC BIT MANIPULATIONS

- 1. Check Odd/Even: `(n & 1) != 0`
- 2. Set Bit `$k$`: `n | (1 << k)`
- 3. Clear Bit `$k$`: `n & ~(1 << k)`
- 4. Toggle Bit `$k$`: `n ^ (1 << k)`

2 ADVANCED BIT TRICKS

- 5. Check Bit `$k$`: `((n >> k) & 1) != 0`
- 6. Clear Lowest Set Bit: `n & (n - 1)`
- 7. Extract Lowest Set Bit: `n & -n`
- 8. Check Power of Two: `n > 0 && (n & (n - 1)) == 0`

⚠️ OPERATOR PRECEDENCE BUG: In Java, `==` has HIGHER precedence than `&`! Always write `(n & 1) == 0` with parentheses around `n & 1`!

💡 AHA MOMENT #4: XOR PAIR CANCELLATION PROPERTY

Because $x \oplus x = 0$ and $x \oplus 0 = x$, XORing all elements in an array where every number appears twice EXCEPT one will ****cancel out all duplicates****, leaving **ONLY** the unique number in $O(N)$ time and $O(1)$ space!

🔪 SINGLE NUMBER (LC 136) $O(N)$ TIME, $O(1)$ SPACE

```
public int singleNumber(int[] nums) {
    int xor = 0;
    for (int num : nums) xor ^= num; // Duplicate numbers cancel out to 0!
    return xor;
}
```

🔪 MISSING NUMBER (LC 268) XOR TRICK

```
public int missingNumber(int[] nums) {
    int xor = nums.length;
    for (int i = 0; i < nums.length; i++) {
        xor ^= i ^ nums[i]; // XOR indices 0..N with array values!
    }
    return xor; // Remaining value is the missing number!
}
```

🔪 KEY TAKEAWAY: XOR eliminates duplicate pairs without needing HashMap or HashSet memory storage!

💡 **AHA MOMENT #5: INTEGER AS A DYNAMIC SET (BITMASK)**

An integer 'mask' represents a set of items up to size 32:

- Bit 'i' is '1' → Element 'i' is INCLUDED in the subset.
- Bit 'i' is '0' → Element 'i' is EXCLUDED.

Iterating 'mask' from '0' to '(1 << N) - 1' generates all 2^N subsets iteratively without recursion!

🔧 **ITERATIVE SUBSET GENERATION USING BITMASK (LC 78)**

```
public List<List<Integer>> subsets(int[] nums) {
    List<List<Integer>> res = new ArrayList<>();
    int totalSubsets = 1 << nums.length; // 2^N
    for (int mask = 0; mask < totalSubsets; mask++) {
        List<Integer> subset = new ArrayList<>();
        for (int i = 0; i < nums.length; i++) {
            if ((mask & (1 << i)) != 0) subset.add(nums[i]); // Bit i is set!
        }
        res.add(subset);
    }
    return res;
}
```

★ **KEY TAKEAWAY:** Bitmasking replaces recursive call stack memory with a simple $O(2^N)$ integer loop!

💡 **AHA MOMENT #6: COUNTING BITS DP RECURRENCE** ('dp[i] = dp[i >> 1] + (i & 1)')

The number of 1-bits in integer 'i' is equal to the number of 1-bits in 'i >> 1' plus '1' if 'i' is odd ('i & 1')!

1 COUNTING BITS DP (LC 338)

```
public int[] countBits(int n) {
    int[] dp = new int[n + 1];
    for (int i = 1; i ≤ n; i++) {
        dp[i] = dp[i >> 1] + (i & 1); // Right-shift + LSB
    }
    return dp;
}
```

2 JAVA BUILT-IN 'INTEGER' BIT APIS

- `Integer.bitCount(n)` → Count set bits.
- `Integer.highestOneBit(n)` → Keep only MSB.
- `Integer.lowestOneBit(n)` → Keep only LSB.
- `Integer.numberOfLeadingZeros(n)` → Leading zeroes.

🚀 **KEY TAKEAWAY:** Native Java bit methods like 'Integer.bitCount()' are optimized into single CPU instructions ('POPCNT')!

💡 AHA MOMENT #7: ADDITION WITHOUT '+' OR '-' OPERATORS

Binary addition consists of two parts:

1. **Sum without Carry:** `a ^ b` (XOR adds bits without carrying over).
2. **Carry:** `(a & b) << 1` (AND detects carry bits, left-shifted by 1).

Repeat until `carry == 0`!

🔗 SUM OF TWO INTEGERS (LC 371) IMPLEMENTATION

```
public int getSum(int a, int b) {
    while (b != 0) {
        int carry = (a & b) << 1; // Calculate carry bits
        a = a ^ b;                // Calculate sum without carry
        b = carry;                // Carry becomes new b
    }
    return a;
}
```

🔑 KEY TAKEAWAY: `a ^ b` computes addition modulo 2, while `(a & b) << 1` propagates carry overflow!

BIT MANIPULATION INTERVIEW DECISION TREE (MERMAID)

PRINTABLE BIT MANIPULATION CHEAT SHEET

Trick / Operation	Code Expression	Explanation & Use Case
Check Odd/Even	<code>(n & 1) != 0</code>	Checks lowest bit (1 = Odd, 0 = Even)
Set Bit <code>\$\$</code>	<code>n (1 << k)</code>	Forces bit <code>\$\$</code> to '1'
Clear Bit <code>\$\$</code>	<code>n & ~(1 << k)</code>	Forces bit <code>\$\$</code> to '0'
Toggle Bit <code>\$\$</code>	<code>n ^ (1 << k)</code>	Flips bit <code>\$\$</code> ('0' → '1', '1' → '0')
Clear Lowest 1-Bit	<code>n & (n - 1)</code>	Flips rightmost set bit to '0' (Kernighan)
Extract Lowest 1-Bit	<code>n & -n</code>	Isolates rightmost set bit using Two's Comp
Power of Two Check	<code>n > 0 && (n & (n - 1)) == 0</code>	'true' if <code>n</code> has EXACTLY one '1' bit


FAANG COACH TIP: Print this page and review it 10 minutes before your technical interview!

1. FUNDAMENTAL BIT TRICKS SUITE

```
boolean checkBit(int n, int k) { return ((n >> k) & 1) != 0; }
int setBit(int n, int k) { return n | (1 << k); }
int clearBit(int n, int k) { return n & ~(1 << k); }
int toggleBit(int n, int k) { return n ^ (1 << k); }
```

2. POWER OF TWO & FOUR TEMPLATE

```
boolean isPowerOfTwo(int n) {
    return n > 0 && (n & (n - 1)) == 0;
}
boolean isPowerOfFour(int n) {
    return n > 0 && (n & (n - 1)) == 0 && (n & 0x55555555) != 0;
}
```

 **REUSABLE CODE:** Copy-paste these templates directly into your interview whiteboard or IDE!

3. BRIAN KERNIGHAN BIT COUNT TEMPLATE

```
int countSetBits(int n) {
    int count = 0;
    while (n != 0) {
        n &= (n - 1); // Clears lowest set bit!
        count++;
    }
    return count;
}
```

4. BITMASK SUBSET ENUMERATION TEMPLATE

```
void enumerateSubsets(int[] nums) {
    int total = 1 << nums.length;
    for (int mask = 0; mask < total; mask++) {
        for (int i = 0; i < nums.length; i++) {
            if ((mask & (1 << i)) != 0) {
                // Element nums[i] is included in subset!
            }
        }
    }
}
```

5. SUBMASK ITERATION & XOR IN-PLACE SWAP TRICKS

Submask Decrement Iteration:

```
// Iterate over all non-empty submasks of 'mask'
for (int sub = mask; sub > 0; sub = (sub - 1) & mask) {
    // Process submask in O(2^K) time!
}
```

In-Place XOR Swap (Zero Temp Memory):

```
// Swap a and b without temporary variable
a ^= b;
b ^= a;
a ^= b;
```

📌 **KEY TAKEAWAY:** Memorize these core templates to solve 95% of FAANG Bit Manipulation problems!

1. LEETCODE 136 — SINGLE NUMBER

🟢 Easy ★★★★★ XOR Pair Cancellation [Amazon](#) [Meta](#)

Problem:

Every element appears twice except for one unique element. Find it in $O(N)$ time and $O(1)$ space.

```
public int singleNumber(int[] nums) {
    int xor = 0;
    for (int num : nums) xor ^= num;
    return xor;
}
```

2. LEETCODE 191 — NUMBER OF 1 BITS (HAMMING WEIGHT)

🟢 Easy ★★★★★ Brian Kernighan Algorithm [Amazon](#) [Apple](#)

Problem:

Return the number of set bits ('1' bits) in an unsigned integer.

```
public int hammingWeight(int n) {
    int count = 0;
    while (n != 0) {
        n &= (n - 1); // Clears rightmost set bit
        count++;
    }
    return count;
}
```

💡 **AHA MOMENT (LC 191):** `'n & (n - 1)'` loops ONLY as many times as there are set 1-bits, making it faster than checking all 32 bits!

3. LEETCODE 338 — COUNTING BITS

Easy ★★★★★ DP + Bit Shift Relation Amazon Meta

Problem:

Return an array `ans` where `ans[i]` is the number of 1 bits in i for $0 \leq i \leq n$.

```
public int[] countBits(int n) {
    int[] dp = new int[n + 1];
    for (int i = 1; i ≤ n; i++) {
        dp[i] = dp[i >> 1] + (i & 1);
    }
    return dp;
}
```

4. LEETCODE 190 — REVERSE BITS

Easy ★★★★★ 32-Bit Shift Construction Apple

Problem:

Reverse bits of a given 32-bit unsigned integer.

```
public int reverseBits(int n) {
    int res = 0;
    for (int i = 0; i < 32; i++) {
        res = (res << 1) | (n & 1); // Shift res left, append LSB of n
        n >>= 1; // Unsigned right shift n
    }
    return res;
}
```

💡 **AHA MOMENT (LC 190):** Always use unsigned right shift `n >>= 1` to prevent sign-bit extension bugs during bit reversal!

5. LEETCODE 268 — MISSING NUMBER

🟢 Easy ★★★★★ Index-Value XOR Cancellation Amazon Meta

Problem:

Given array of N distinct numbers in range $[0, n]$, return the missing number.

```
public int missingNumber(int[] nums) {
    int xor = nums.length;
    for (int i = 0; i < nums.length; i++) {
        xor ^= i ^ nums[i]; // XOR index and value
    }
    return xor;
}
```

6. LEETCODE 371 — SUM OF TWO INTEGERS

🟡 Medium ★★★★★ Bitwise Addition (No + Operator) Amazon

Problem:

Calculate sum of two integers 'a' and 'b' without using operators '+' and '-'.

```
public int getSum(int a, int b) {
    while (b != 0) {
        int carry = (a & b) << 1;
        a = a ^ b;
        b = carry;
    }
    return a;
}
```

💡 **AHA MOMENT (LC 268):** XORing array values with their 0-to-N indices cancels out all existing pairs, leaving ONLY the missing number!



7. LEETCODE 201 — BITWISE AND OF NUMBERS RANGE

Medium ★★★★★ Common Binary Prefix Search Amazon Meta

Problem:

Return bitwise AND of all numbers in range `[left, right]` inclusive.

```
public int rangeBitwiseAnd(int left, int right) {
    int shifts = 0;
    while (left != right) { // Find common binary prefix!
        left >>= 1;
        right >>= 1;
        shifts++;
    }
    return left << shifts;
}
```

8. LEETCODE 78 — SUBSETS (BITMASK APPROACH)

Medium ★★★★★ Bitmask Iteration Amazon

Problem:

Return power set using iterative $0 \dots 2^N - 1$ bitmasking.

```
public List<List<Integer>> subsets(int[] nums) {
    List<List<Integer>> res = new ArrayList<>();
    int total = 1 << nums.length;
    for (int mask = 0; mask < total; mask++) {
        List<Integer> subset = new ArrayList<>();
        for (int i = 0; i < nums.length; i++) {
            if ((mask & (1 << i)) != 0) subset.add(nums[i]);
        }
        res.add(subset);
    }
    return res;
}
```

🚩 **KEY TAKEAWAY:** LC 201 finds the common binary prefix of `left` and `right` because non-matching trailing bits flip to 0 during range ANDing!

9. LEETCODE 260 — SINGLE NUMBER III

Medium
★★★★★
Lowest Set Bit Partitioning
Amazon
Google

Problem:
Exactly two elements appear once, all others appear twice. Find the two unique elements in $O(N)$ time and $O(1)$ space.

```

public int[] singleNumber(int[] nums) {
    int xor = 0;
    for (int num : nums) xor ^= num; // xor = num1 ^ num2
    int diffBit = xor & -xor;        // Extract lowest set bit where
    num1 & num2 differ!
    int[] res = new int[2];
    for (int num : nums) {
        if ((num & diffBit) == 0) res[0] ^= num; // Group 1
        else res[1] ^= num;                    // Group 2
    }
    return res;
}
```

10. LEETCODE 421 — MAXIMUM XOR OF TWO NUMBERS (TRIE INTEGRATION)

Hard
★★★★★
31-Bit Binary Trie Search
Amazon
Google

Problem:
Maximum result of `nums[i] ^ nums[j]`. (Refer to **Topic 11: Trie Masterclass** for full Bitwise Trie implementation!).

```

// 31-Bit Binary Trie Insert & Max Query:
// Insert each integer bit-by-bit from bit 31 down to 0.
// For query x, greedily pick opposite bit (1 - bit) in Trie to maximize XOR!
// Time: O(32 * N) = O(N), Space: O(32 * N)
```

🧠 **FAANG COACH TIP:** `diffBit = xor & -xor` isolates a bit position where `num1` has `1` and `num2` has `0`, splitting array into 2 independent XOR buckets!

★ THE 4 GOLDEN LAWS OF BIT MANIPULATION

- Kernighan Law:** `n & (n - 1)` clears lowest set 1-bit in $O(1)$ constant time.
- Lowest Set Bit Law:** `n & -n` isolates the rightmost 1-bit using Two's Complement.
- XOR Cancellation Law:** `x ^ x = 0` and `x ^ 0 = x` eliminate duplicate pairs.
- Unsigned Shift Law:** Always use `>>>` (unsigned right shift) to prevent sign bit extension bugs!

🚫 COMMON BIT BUGS & PITFALLS

- Operator Precedence:** `==` has higher precedence than `&!` Always write `!(n & 1) == 0`.
- Shift Overflow:** `1 << 31` creates negative integer! Use `1L << k` for long 64-bit masks.

📁 NEETCODE & BLIND 75 CONFIDENCE TRACKER

🟢 Easy LC 136 Single Number	🟢 Easy LC 191 Number of 1 Bits	🟢 Easy LC 338 Counting Bits
🟢 Easy LC 190 Reverse Bits	🟢 Easy LC 268 Missing Number	🟡 Med LC 371 Sum Two Integers
🟡 Med LC 201 Bitwise AND Range	🟡 Med LC 78 Subsets Bitmask	🟡 Med LC 260 Single Number III
🔴 Hard LC 421 Bitwise Trie Max XOR		

👉 **Next Master Topic: Topic 18** → **Math Basics Masterclass**

🚩 **TOPIC 17 BIT MANIPULATION MASTERCLASS COMPLETE!** Turn the page to **Topic 18: Math Basics Masterclass!**